TÉCNICO
UNIVERSIDADE
DE LISBOA

CLOUD COMPUTING AND VIRTUALIZATION

MECD – 2nd Semester 2025/2026

Nature@Cloud

Final Report

Group 26

Authors

Catarina Gonçalves (103989)
Vasco Fernandes (106176)
Maria Rodrigues (116489)

Instituto Superior Técnico
Universidade de Lisboa

I. Introduction

This project deploys Nature@Cloud on Amazon Web Services (AWS), which supports CPU-intensive fractal generation, Gray-Scott simulation, and DNA visualization workloads. These requests have different computational costs, since their execution cost depends on the workload type and the input parameters. Therefore, the goal is not only to execute requests correctly, but also to reduce latency and infrastructure cost by combining EC2 workers with AWS Lambda [1, 2]. In this architecture, Lambda is used as a fallback backend when no EC2 worker is available and as an overflow backend when the predicted EC2 completion time is too high [2, 3].

Measured execution time alone is not a reliable indicator of request complexity. The same request may take longer when executed on a busy worker, even if the amount of computation required by the request has not changed. For this reason, scheduling and scaling decisions should not only rely on observed response time. Our system estimates the complexity of requests using Javassist bytecode-level instrumentation and stores request-level execution metrics in DynamoDB. These historical measurements are then used to predict the expected cost of future requests from their input parameters.

The system uses a custom Java load balancer that discovers EC2 workers in the `ACTIVE` state, estimates the cost of incoming requests, and selects the worker with the lowest predicted completion score. When no EC2 worker is available, or when EC2 execution is predicted to be inefficient, the request can be redirected to Lambda. The same estimated-load information is also used by a custom auto-scaling component to adjust the number of EC2 workers according to the current backlog.

The report describes the system architecture, instrumentation and metric collection, metric storage, request cost estimation, load balancing with Lambda fallback and overflow, auto-scaling, fault tolerance, and evaluation results.

II. System Architecture

Nature@Cloud is a cloud service that executes computationally intensive workloads inspired by natural processes: Julia-set fractals, Gray-Scott reaction-diffusion simulations, and DNA sequence matching. The architecture is centered on a dedicated Controller EC2 instance, which runs the custom Java load balancer and the custom auto-scaling logic. External clients send requests to this controller, which then routes each request to an EC2 worker from the Auto Scaling Group (ASG) or to an AWS Lambda function [4, 2].

The EC2 worker pool is the primary execution backend. Each worker runs the instrumented Nature@Cloud web server and executes the original workload handlers. The controller periodically discovers active EC2 workers in the ASG, maintains their current state, and tracks the estimated amount of work already assigned to each. This allows the load balancer to make scheduling decisions based not only on worker availability, but also on the expected computational cost of the new request and on the current estimated load of each worker.

AWS Lambda is used as a secondary execution backend, providing a fallback path when no EC2 worker is available and an overflow path when the EC2 pool is overloaded or predicted to provide poor response time [2]. This gives the system an additional execution option during load spikes or temporary worker unavailability, while keeping the reusable EC2 worker pool as the normal execution path.

The repository is organized into workload, instrumentation, web server, controller, metrics, and deployment components. The `dna`, `fractals`, and `grayscott` modules contain the original Nature@Cloud workloads. The `javassist` module contains the provided bytecode instrumentation tools and the custom `RequestICount` tool, which records request-level instruction counts. The `webserver` module contains both the instrumented worker server and the controller-side logic.

Inside `webserver`, the main implementation is divided into the following sub-packages:

- `cost/`: contains the request cost estimators. These classes estimate the expected computational cost of incoming requests from their URL parameters, using either workload-specific fallback models or historical measurements stored in DynamoDB.
- `lb/`: contains the custom load-balancing and controller logic. This includes the HTTP forwarding path, EC2 worker discovery, worker-state tracking, request scheduling, Lambda invocation, and ASG capacity control.
- `as/`: contains the auto-scaling logic used by the controller to decide when the EC2 worker pool should grow or shrink. The scaling decision is based on the estimated load assigned to active workers and the current ASG capacity.
- `metrics/`: contains the metrics storage abstraction and its implementations. The system uses DynamoDB as the persistent repository for request-level execution metrics, while the in-memory implementation is kept as a local or fallback variant.

The `scripts` directory contains the deployment and operational scripts used to build and run the system on AWS. These scripts cover virtual machine (VM) installation, Amazon Machine Image (AMI) creation, controller launch, ASG deployment, AWS Lambda deployment, Identity and Access Management (IAM) setup, testing, and cleanup [4, 2]. They support the end-to-end deployment of the controller, the EC2 worker pool, and the Lambda backend.

III. Instrumentation and Metrics Collection

The system uses Javassist to collect request-level execution metrics from the Nature@Cloud workloads [5]. The custom `RequestICount` tool adds a counting logic to the workload code, allowing the system to record, for each request, the number of executed methods, executed basic blocks, and executed bytecode instructions. These metrics are collected alongside elapsed execution time, allowing the system to characterize the computational work performed by a request rather than only the time it took to complete.

Since the web server may process multiple requests simultaneously, metric counters must be isolated per request. This is achieved by storing counters in a `ConcurrentHashMap<Long, MutableMetrics>`, indexed by the identifier of the current execution thread [6]. Each request handler resets the counters associated with its thread before executing the workload, reads the collected values after completion, and clears the thread state before returning. This prevents metrics of simultaneous requests from being mixed when multiple handlers execute at the same time.

Instrumentation is applied to the packages that implement the three supported workloads. Each workload is exposed through a measured handler that preserves the original request logic while adding metric collection around it. The handler flow is the same for fractals, Gray-Scott, and DNA requests: initialize the metrics for the current thread, execute the original workload, read the col-

lected counters, record the completed request, and, finally, clear the per-thread state. Besides the Javassist counters, each record also includes the workload name, the raw query parameters, the request status, possible error information, and elapsed execution time measured with `System.nanoTime()`.

The collected instruction and basic-block counts are useful because they are more stable indicators of intrinsic request complexity than observed response time alone. Observed response time is affected by factors that are not part of the request itself, such as the current load of the worker or interference from other concurrent requests. In contrast, bytecode-level counters are tied to the actual execution path of the workload, so they provide a more direct measure of the computational work performed. Therefore, the instrumentation provides the raw data to be later used by the request cost estimator, load balancer, and auto-scaling logic.

The selected metrics also reflect an overhead trade-off. Counting every bytecode instruction, basic block, and method execution increases the cost of each instrumented request, so the system does not attempt to collect more detailed low-level events such as individual memory accesses. The chosen counters provide enough information to estimate computational complexity while keeping the instrumentation logic simple and directly usable by the load balancer and auto-scaler.

For local observability and debugging, each worker also appends the collected metrics to per-workload CSV files. Successful completed-request records are then submitted to the metrics repository, where they can be reused by the cost estimator when predicting the cost of future requests.

IV. Metrics Storage and Data Structures

The system separates two types of information: historical request metrics and runtime worker state. Historical request metrics describe completed executions and are used to estimate the cost of future requests. Runtime worker state describes the current condition of the EC2 worker pool and is used by the load balancer and auto-scaler when assigning new requests or changing the desired capacity of the Auto Scaling Group (ASG). This separation is important because historical metrics must persist across executions, while runtime scheduling state changes frequently and should be updated directly in memory rather than through repeated database operations.

At the worker level, each measured handler records completed executions in a workload-specific CSV file: `metrics-fractals.csv`, `metrics-grayscott.csv`, and `metrics-dna.csv`. Each row stores the workload name, raw query string, thread identifier, method count, basic-block count, bytecode instruction count, elapsed execution time in nanoseconds, request status, and possible error message. These files provide a local execution trace for each worker, making it easier to inspect the measurements produced by the instrumentation and to compare requests with different input parameters during profiling.

In addition to the local CSV files, completed-request profiles are stored in the `NatureCloudMetrics` table in DynamoDB [7]. Each item stores the workload, query string, worker identifier, estimated instruction count, measured instruction count, elapsed time, completion timestamp, thread identifier, method count, basic-block count, request status, success flag, and error message. DynamoDB provides a shared persistent history across workers, allowing the cost estimator to reuse previous executions instead of relying only on the local files of a single worker. DynamoDB persistence is controlled by `CNV_METRICS_DYNAMODB_ENABLED`. When it is disabled the repository still records recent metrics in its local in-memory cache.

Metric storage is not placed on the critical path of request correctness. If storing a metric record in DynamoDB fails temporarily, the handler still returns the workload result to the client whenever the computation itself completed successfully. The metrics repository also keeps a small in-memory cache of recently recorded metrics, allowing recent averages to be computed without querying DynamoDB on every estimation decision.

The controller maintains a separate online view of the EC2 worker pool through the `WorkerRegistry`. Each worker is represented by a `WorkerState`, which stores the worker identifier, base URL, lifecycle state, number of in-flight requests, current estimated instruction load, number of completed and failed requests, and estimated processing speed in instructions per second. This state is kept in memory since it is updated whenever requests are assigned, completed, or fail.

The worker lifecycle is represented by `WorkerLifecycle`. Workers in the `ACTIVE` state can receive new requests during normal scheduling. Workers in the `DRAINING` state are no longer preferred for new assignments, but may still be tracked until their current in-flight requests finish. Workers marked as `UNHEALTHY` are excluded from normal scheduling. Keeping this runtime state in the controller avoids using DynamoDB as a high-frequency coordination mechanism and allows the scheduler and auto-scaler to react immediately to changes in the estimated worker load.

V. Request Cost Estimation

The load balancer and auto-scaler require an estimate of the cost of each request before the request is executed. Since the instrumentation measures request complexity in bytecode instructions, the controller also represents predicted cost as an estimated number of executed instructions. This gives the system a common unit for comparing requests from different workloads, namely fractals, Gray-Scott, and DNA.

The first estimation layer consists of workload-specific fallback models derived from profiling data collected with the instrumented workers. These models use input parameters that reflect the dominant loop structure of each workload. Fractal generation depends mainly on the image area and the number of iterations per pixel. Gray-Scott simulation depends on the simulation grid size and on the maximum number of iterations. DNA comparison depends mainly on the product of the two sequence lengths, with adjustments to parameters that change the matching behavior.

The resulting fallback models are:

$$\hat{I}_{\text{fractals}} = \texttt{w} \cdot \texttt{h} \cdot (734.308 + 7.093 \cdot \texttt{iterations}),$$
$$\hat{I}_{\text{gray-scott}} = 273.192 \cdot \texttt{size}^2 \cdot \texttt{maxIterations},$$
$$\hat{I}_{\text{dna}} = \texttt{len1} \cdot \texttt{len2} \cdot (0.715 + 0.216 \cdot \texttt{knownSimilar} + 1.463 \cdot \mathbb{1}_{\{\texttt{minLength} \geq 20\}} - 0.511 \cdot \texttt{stopOnFirst}).$$

Here, $\hat{I}$ denotes the estimated number of executed bytecode instructions. The Boolean parameters are interpreted as indicator variables, with value 1 when the condition is true and 0 otherwise. These models were fitted by linear regression on profiling records from different parameter configurations [8], obtaining $R^2 = 1.000$ for Fractals and Gray-Scott, and $R^2 = 0.986$ for DNA. The fitted relations are

visually validated by the regression plots provided in Appendix A. The models are used as fallback estimators when the system has no useful historical information for a request.

The deployed estimator then refines this fallback estimate using the historical metrics stored in DynamoDB. For each workload, `DynamoDbBackedCostEstimator` periodically builds a snapshot of recent successful executions. When a new request arrives, the estimator first computes a canonical representation of its parameters and checks whether the same parameter set has already been observed. If an exact historical match exists, it uses the average measured instruction count for that request.

If no exact match exists, the estimator uses recent executions of the same workload. It computes the average ratio between measured instructions and a workload-specific feature unit, such as $\texttt{w} \cdot \texttt{h} \cdot \texttt{iterations}$ for fractals, $\texttt{size}^2 \cdot \texttt{maxIterations}$ for Gray-Scott, and $\texttt{len1} \cdot \texttt{len2}$ for DNA. This ratio is then multiplied by the feature value of the new request. If neither an exact match nor enough same-workload history is available, the estimator falls back to the static workload-specific model.

This logic can be summarized as:

$$\hat{I}(r) = \begin{cases} \overline{I}(r), & \text{if an exact historical match exists,} \\ \overline{\rho}_w \cdot f_w(r), & \text{if recent history exists for workload } w, \\ S_w(r), & \text{otherwise,} \end{cases}$$

where $\hat{I}(r)$ is the estimated instruction count for a request r, $\overline{I}(r)$ is the average measured instruction count for the same request, $f_w(r)$ is the workload-specific feature function, $\overline{\rho}_w$ is the average number of instructions per feature unit for a workload w, and $S_w(r)$ is the fallback estimator for that workload.

This hybrid approach gives the controller a prediction even for unseen requests while allowing the estimator to adapt as more executions are observed during operation.

VI. Task Scheduling, Load Balancing, and Lambda Fallback

The load balancer receives each client request, parses the workload parameters, and estimates the expected instruction cost before selecting a backend. It then obtains the EC2 workers currently in the `ACTIVE` state from the `WorkerRegistry` and applies an instruction-aware scheduling policy. Instead of selecting the worker with the smallest number of active requests, the scheduler considers the estimated instruction backlog already assigned to each worker and the estimated processing speed of that worker.

For a worker w and request r, the predicted completion score is based on

$$T(w, r) = \frac{L_w + \hat{I}(r)}{S_w},$$

where L_w is the estimated number of instructions already assigned to worker w, $\hat{I}(r)$ is the estimated instruction cost of the new request, and S_w is the worker's estimated processing speed in instructions per second. The selected worker is the worker in the `ACTIVE` state with the smallest score. The scheduler also adds a small penalty proportional to the number of in-flight requests, avoiding excessive concurrency on a single instance. For very large requests, an additional penalty is added when the worker is already executing other requests. This reduces the probability of assigning several heavy requests to the same worker only because its estimated instruction backlog is slightly lower.

After a worker is selected, the controller immediately updates its `WorkerState` by increasing the in-flight request count and adding the request estimate to the current instruction load. This update is performed before forwarding the request, so concurrent scheduling decisions observe the new predicted load. When the backend replies successfully, the request is marked as completed and the worker speed estimate is updated using an exponential moving average of the observed processing rate. If forwarding fails or the worker returns a server error, the assigned load is removed, the failure counter is updated, and the worker is marked as `UNHEALTHY` before another backend is attempted. EC2 forwarding is retried only for a bounded number of attempts, avoiding indefinite retry loops when the worker pool is failing.

AWS Lambda is controlled by `CNV_LAMBDA_ENABLED`, with the main Lambda and auto-scaling configuration parameters summarized in Appendix A, Table I. When enabled, Lambda can be used in three cases: when no EC2 worker in the `ACTIVE` state is available, when the predicted completion time of EC2 exceeds the configured overflow threshold and the overflow policy allows Lambda to be used, or when EC2 forwarding fails [2, 3]. By default, the overflow path is only considered when the selected EC2 worker already has in-flight requests. This behavior is controlled by `CNV_LAMBDA_OVERFLOW_ONLY_WHEN_EC2_BUSY`. A request is only eligible for Lambda if its estimated instruction count is below the configured maximum, preventing very large requests from being sent to Lambda. The overflow delay threshold was set to favor EC2 during normal operation and to use Lambda only when the predicted EC2 delay is large enough to justify the additional backend invocation. EC2 remains the preferred backend, since instances are already running and can serve multiple requests over time, while Lambda is used to reduce tail latency during bursts, EC2 startup periods, or worker failures.

Lambda executions are used as an alternative execution path and are not part of the same Javassist-based metrics collection pipeline as EC2 workers. Consequently, the historical cost model is primarily learned from instrumented EC2 executions, while Lambda is used operationally to absorb bursts, cover EC2 startup delays, and mask worker failures.

Algorithm 1 summarizes the scheduling and fallback logic implemented by the controller.

VII. Auto-Scaling Algorithm

The system uses a custom Java auto-scaler to adjust the number of EC2 workers in the Auto Scaling Group (ASG) according to the estimated instruction load maintained by the controller [1, 4]. At each control interval, the auto-scaler reads the workers in the `ACTIVE` state from the `WorkerRegistry`, obtains the current desired ASG capacity, and computes the average estimated load per worker in the `ACTIVE` state:

$$\overline{L} = \frac{\sum_{w \in W_{\texttt{ACTIVE}}} L_w}{|W_{\texttt{ACTIVE}}|},$$

where L_w is the estimated number of instructions currently assigned to worker w. If there are no workers in the `ACTIVE` state, the average load is treated as zero for that interval. The custom auto-scaler can be enabled or disabled through `CNV_AS_ENABLED`.

Algorithm 1 Load balancing and Lambda fallback

parse request parameters
$\hat{I}(r) \leftarrow$ estimate request instructions
$W \leftarrow$ EC2 workers in state `ACTIVE`
$w^\star \leftarrow \arg\min_{w \in W} T(w, r)$, if $W \neq \emptyset$
if $W = \emptyset$ **then**
    if Lambda can execute r **then**
        invoke Lambda as fallback
    else
        return service unavailable
    end if
else if Lambda can execute r and $T(w^\star, r)$ exceeds overflow threshold **then**
    if Lambda invocation succeeds **then**
        return Lambda response
    else
        continue with EC2 forwarding
    end if
end if
for each forwarding attempt **do**
    $W \leftarrow$ EC2 workers in state `ACTIVE`
    if $W = \emptyset$ **then**
        break
    end if
    choose $w^\star = \arg\min_{w \in W} T(w, r)$
    assign request to $w^\star$ and update in-flight load
    forward request to $w^\star$
    if worker response is successful **then**
        mark request as completed and update worker speed estimate
        return worker response
    else
        remove assigned load and mark worker as `UNHEALTHY`
    end if
end for
if Lambda can execute r **then**
    invoke Lambda as failure fallback
else
    return service unavailable
end if

Scale-out and scale-in are controlled by two configurable thresholds over this average estimated backlog. In Algorithm 2, these thresholds are denoted by H_{high} and H_{low}. Their values are read from `CNV_AS_HIGH_AVG_LOAD` and `CNV_AS_LOW_AVG_LOAD`, respectively. If these variables are not overridden at deployment time, the auto-scaler uses the preset values 2.5×10^9 and 2.5×10^8 estimated instructions. The minimum and maximum number of workers, control interval, consecutive-interval requirements, and cooldown duration are also configurable through environment variables. The main deployment parameters are summarized in Appendix A, Table I.

The concrete threshold values were chosen empirically to make the controller react to sustained backlog rather than to isolated expensive requests. The high threshold triggers scale-out when the average assigned load per active worker becomes comparable to several medium-size requests, while the lower threshold is one order of magnitude smaller to create hysteresis and avoid immediate scale-in after a temporary load drop. The 15-second control interval and 120-second cooldown were selected to give AWS time to launch or terminate instances before another scaling decision is applied.

Algorithm 2 summarizes the auto-scaling control loop.

Before evaluating a new scaling action, the auto-scaler first processes workers previously marked as `DRAINING`. A draining worker is no longer selected for normal scheduling, but it is not terminated while it still has in-flight requests. Once its in-flight request count reaches zero, the auto-scaler terminates the EC2 instance through the AWS Auto Scaling API and decrements the ASG desired capacity.

Algorithm 2 Auto-scaling control loop

process workers already marked as `DRAINING`
obtain current ASG desired capacity d
$W_{\texttt{ACTIVE}} \leftarrow$ workers in the `ACTIVE` state from `WorkerRegistry`
compute $\overline{L}$, the average estimated load per worker in the `ACTIVE` state
if cooldown is active **then**
    skip new scaling action for this interval
else if $W_{\texttt{ACTIVE}} = \emptyset$ and $d < d_{\min}$ **then**
    request desired capacity $d_{\min}$
    reset counters and start cooldown
else if $\overline{L} \geq H_{\text{high}}$ **then**
    increment high-load counter and reset low-load counter
    if high-load counter reaches N and $d < d_{\max}$ and no capacity increase is pending **then**
        request desired capacity $d + 1$
        reset counters and start cooldown
    end if
else if $\overline{L} \leq H_{\text{low}}$ **then**
    increment low-load counter and reset high-load counter
    if low-load counter reaches M, $d > d_{\min}$, and $|W_{\texttt{ACTIVE}}| > d_{\min}$ **then**
        select an idle worker in the `ACTIVE` state
        if such worker exists **then**
            mark worker as `DRAINING`
            reset counters
        end if
    end if
else
    reset load counters
end if

Scale-out is triggered only after the average load of `ACTIVE` workers remains above the high threshold for N consecutive intervals. The desired capacity is increased by one only if the maximum capacity has not been reached and there is no previously requested capacity increase still pending. This avoids repeatedly launching instances before AWS has had time to make the previous scale-out effective.

Scale-in is considered only after the average load of `ACTIVE` workers remains below the low threshold for M consecutive intervals, the current desired capacity is above the minimum, and the number of `ACTIVE` workers is also above the minimum. The auto-scaler then selects an idle worker in the `ACTIVE` state and marks it as `DRAINING`. This avoids interrupting long-running requests and prevents the load balancer from assigning new work to an instance that is being removed.

The combination of consecutive-interval counters, cooldown periods, pending-capacity checks, and draining reduces oscillation [9]. This is important because worker startup and termination are not instantaneous, and reacting to every short load fluctuation would create unstable scaling behavior.

VIII. Fault Tolerance

The Controller is a critical component of the Nature@Cloud architecture as all external requests go through it before reaching an execution backend. Its fault-tolerance logic focuses on detecting failures in EC2 workers, avoiding repeated use of failed instances, and retrying requests on an alternative backend whenever possible. This does not remove all possible failures from the system, since the Controller itself remains a single point of failure, but it can prevent individual worker failures from immediately becoming client-visible errors.

Fault detection is performed during request forwarding. The `ForwardingHandler` wraps outbound communication with EC2 workers in exception-handling logic and uses a request timeout when waiting for the worker response. If a worker becomes unreachable,

crashes during execution, returns a server-side error, or fails to respond within the allowed time, the Controller treats the forwarding attempt as failed. The estimated load previously assigned to that worker is then removed from its `WorkerState`, so that the internal load view remains consistent with the requests that are still considered in progress.

After a forwarding failure, the corresponding worker is marked as `UNHEALTHY` in the `WorkerRegistry`, consequently being removed from normal scheduling, since only `ACTIVE` workers are selected. The controller then retries the request on another available EC2 worker, up to a bounded number of forwarding attempts. If no suitable EC2 worker is available, or if EC2 forwarding continues to fail, the controller may invoke AWS Lambda as a failure fallback, provided that Lambda is enabled and that the request satisfies the configured eligibility limits.

The same lifecycle mechanism is also used during intentional scale-in. When the auto-scaler decides to remove capacity, it does not immediately terminate a worker that may still be executing a request. Instead, it marks an idle worker as `DRAINING`. Workers in the `DRAINING` state are excluded from new scheduling decisions, but remain tracked until their in-flight request count reaches zero. Only then does the auto-scaler terminate the EC2 instance. This avoids interrupting long-running requests during scale-in and maintains both load-balancing and auto-scaling policies consistent.

The controller also keeps its view of the EC2 pool synchronized with AWS. The `AwsWorkerDiscovery` component periodically queries the ASG and updates the `WorkerRegistry` with the instances that are currently part of the worker pool [4]. If an instance disappears from the ASG, for example because of termination or external intervention, it is either marked as `UNHEALTHY` if it still has in-flight requests, or removed from the registry if it is idle. Conversely, an instance that was previously marked as `UNHEALTHY` can return to the `ACTIVE` state if it is later rediscovered as part of the ASG.

Overall, the controller handles worker failures by detecting failed forwarding attempts, removing the failed worker from normal scheduling, and retrying the request through another EC2 worker or through Lambda when possible. If no valid backend can execute the request, the controller returns an explicit service error instead of silently dropping the request.

IX. Evaluation and Results

The evaluation uses three sources of evidence: the regression plots used to validate the static fallback models, the telemetry records exported from deployed executions, and the controller logs used to inspect auto-scaling behavior. The evaluation is therefore focused on validating the main mechanisms of the system rather than providing a complete performance benchmark against alternative schedulers.

A. Fallback Model Validation

The static fallback models introduced in Section V. were fitted from profiling records and are visually summarized by the regression plots in Appendix A, Figure 1 [8]. The plots show that the fitted models capture the main relation between input parameters and measured bytecode instruction counts, especially for the Fractals and Gray-Scott workloads. These models are therefore suitable as fallback estimators when no exact historical match is available in DynamoDB.

B. Runtime Cost Estimation

Appendix A, Table III shows selected examples from the telemetry data. APE denotes absolute percentage error, computed as $|\hat{I} - I|/I \times 100$. Since this measures error relative to the measured value, it can exceed 100% when the estimate is more than twice the actual instruction count.

The examples show that the estimator can be highly accurate for some parameter configurations, such as the large Fractals request and the regular Gray-Scott cases. However, the table also shows large errors for some Fractals and DNA requests. These mismatches indicate that the deployed estimator is not uniformly accurate across all workloads and parameter combinations, especially when there is little historical data for a given request pattern.

The largest errors occur when the request is either poorly represented in the historical records or when the workload has parameters whose effect is not fully captured by the static fallback model. In those cases, the estimator may still provide a useful coarse ordering of the request cost, but it is less reliable as an absolute predictor of the final instruction count.

Appendix A, Table II summarizes the same telemetry records by workload. Gray-Scott is the most stable workload, with a median absolute percentage error of 0.0% and 96.0% of its records within 10% error. Fractals and DNA have higher median errors, showing that the current estimator is more sensitive to parameter variation and missing historical matches for these workloads.

Overall, the telemetry confirms that bytecode-level instruction counts provide a useful common unit for comparing heterogeneous requests, while also showing that the deployed estimator still needs improvement for less regular workloads and poorly represented parameter configurations. Even when the absolute estimate is inaccurate, the instruction estimate remains useful to the scheduler when it separates clearly small requests from substantially heavier ones, since the load-balancing decision depends mainly on relative expected load.

C. Autonomous Scaling Behaviour

The controller logs were used to verify that the custom auto-scaler reacted to changes in the estimated EC2 backlog. During idle periods, the system maintained the minimum worker capacity. When the workload burst started, the estimated average load of `ACTIVE` workers increased above the scale-out threshold, causing the controller to request additional ASG capacity. During the sustained burst, the worker pool grew beyond the minimum capacity and new requests were distributed across multiple EC2 workers. After the burst, the estimated load decreased, and the auto-scaler selected idle workers for `DRAINING` before terminating them.

The observed auto-scaling behavior is summarized in Appendix A, Table IV. This qualitative evaluation shows that the implemented control loop followed the intended behavior in the observed run: maintaining the minimum fleet when idle, scaling out under sustained estimated load, and scaling in conservatively after the burst had passed.

An important aspect of the observed scale-in behavior is that workers were not immediately selected for termination while they still had assigned in-flight load. Instead, workers first entered the `DRAINING` state, which prevented the scheduler from assigning new requests to them and allowed already assigned requests to finish before termination.

X. Limitations and Conclusion

Although the system implements the main mechanisms required for application-aware scheduling and scaling, some limitations remain. First, request cost estimation depends on the amount and diversity of the profiling data available to the system. This is also the main limitation of the evaluation dataset: most collected records correspond to Gray-Scott executions, while DNA and Fractals have fewer examples. As a result, telemetry is useful for validating that the metrics pipeline works and to inspect estimator behavior, but does not provide equally strong evidence for all workloads. When the system starts with little historical data, or when it receives parameter combinations that are not well represented in previous executions, the estimator must rely on the static fallback models. These models capture the main cost trends of each workload, but may still produce local estimation errors for unseen configurations.

Second, the auto-scaling mechanism relies on the estimated instruction backlog maintained by the controller. This makes scaling more application-aware than using only infrastructure-level metrics such as CPU utilization, but it also means that scaling quality depends on the accuracy of the estimator and on the correctness of the in-memory worker state. Moreover, EC2 startup and termination times are not instantaneous, so the system cannot react immediately to sudden load changes. Cooldown periods, consecutive-interval counters, and draining reduce oscillation, but they also introduce a deliberate delay before scaling actions are applied.

Third, Lambda is useful as a fallback and overflow backend, but it is not intended to replace the EC2 worker pool for sustained heavy workloads. Lambda can reduce latency during bursts, worker failures, or EC2 startup periods, but long or repeated use may be more expensive than executing the same workload on already-running EC2 instances [2]. For this reason, the implementation bounds Lambda usage through configuration parameters such as the maximum eligible estimated instruction count.

Finally, DynamoDB provides persistent shared storage for request metrics, but repeatedly querying external storage in the scheduling path could increase latency [7]. This is mitigated by using snapshots and an in-memory cache of recent metrics, while keeping high-frequency runtime state, such as worker load and in-flight request counts, inside the controller. This design reduces database dependency during request scheduling, but it also means that runtime state is not itself durable if the controller fails.

Overall, the system implements an application-aware cloud execution platform for Nature@Cloud. It combines Javassist-based instruction metrics, DynamoDB-backed historical profiling, instruction-aware load balancing, Lambda fallback and overflow, and custom EC2 auto-scaling. Instead of relying only on generic infrastructure metrics or request counts, the controller uses estimated computational cost to route requests and adapt the worker pool. The resulting architecture provides a more informed basis for handling heterogeneous workloads, while still leaving room for future improvements in estimator accuracy, controller fault tolerance, and full end-to-end performance evaluation.

XI. Future Work

Several extensions could improve the current architecture. First, the estimator could be extended with a learned request classification model [8]. Instead of only predicting an instruction count, the controller could classify requests into cost classes such as light, medium, and heavy. These classes could then influence scheduling decisions, for example, by isolating heavy requests on less loaded workers or by delaying them until enough EC2 capacity is available. This would reduce the risk of placing several heavy requests on the same instance and could improve tail latency.

Second, the system could make cost estimation and scaling more adaptive. The current fallback models are fixed after profiling, while future versions could update model coefficients online as new executions are recorded. The auto-scaler could also use recent arrival rates and estimated request costs to anticipate short-term bursts instead of reacting only to the current backlog. This would be useful because EC2 instances take time to start.

Finally, the architecture could be strengthened by replicating the controller and by extending the evaluation. A replicated controller design would reduce the current single point of failure, but would require coordination to keep worker state consistent. A stronger evaluation would compare the implemented instruction-aware scheduler against simpler baselines, such as round-robin, least-connections, and EC2-only execution without Lambda overflow. These baselines would help quantify the benefit of using application-level instruction estimates instead of only worker counts or request counts.

References

[1] Amazon Web Services. *What is Amazon EC2 Auto Scaling?* 2026. URL: https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html (visited on 06/06/2026).

[2] Amazon Web Services. *AWS Lambda Documentation*. 2026. URL: https://docs.aws.amazon.com/lambda/ (visited on 06/06/2026).

[3] Amazon Web Services. *Invoke a Lambda function synchronously*. 2026. URL: https://docs.aws.amazon.com/lambda/latest/dg/invocation-sync.html (visited on 06/06/2026).

[4] Amazon Web Services. *Auto Scaling groups*. 2026. URL: https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-groups.html (visited on 06/06/2026).

[5] Javassist. *Javassist Tutorial*. 2026. URL: https://www.javassist.org/tutorial/tutorial.html (visited on 06/06/2026).

[6] Oracle. *ConcurrentHashMap – Java Platform SE 17*. 2026. URL: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html (visited on 06/06/2026).

[7] Amazon Web Services. *What is Amazon DynamoDB?* 2026. URL: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html (visited on 06/06/2026).

[8] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. 2nd ed. Springer, 2021. DOI: 10.1007/978-1-0716-1418-1.

[9] Amazon Web Services. *Scaling cooldowns for Amazon EC2 Auto Scaling*. 2026. URL: https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-scaling-cooldowns.html (visited on 06/06/2026).

A Additional Configuration and Validation Material

Table I. Main controller configuration parameters used in the final deployment.

Parameter	Value	Purpose
CNV_LAMBDA_ENABLED	true	Enables Lambda fallback and overflow execution
CNV_LAMBDA_MAX_ESTIMATED_INSTRUCTIONS	60000000000	Avoids sending very large requests to Lambda
CNV_LAMBDA_OVERFLOW_THRESHOLD_SECONDS	30	Minimum predicted EC2 delay before considering Lambda overflow
CNV_LAMBDA_OVERFLOW_ONLY_WHEN_EC2_BUSY	true	Considers overflow only when the selected EC2 worker already has in-flight requests
CNV_AS_HIGH_AVG_LOAD	2500000000	Average worker load threshold for scale-out
CNV_AS_LOW_AVG_LOAD	250000000	Average worker load threshold for scale-in
CNV_AS_INTERVAL_SECONDS	15	Auto-scaler control interval
CNV_AS_SCALE_OUT_CONSECUTIVE	2	Number of consecutive high-load intervals required before scale-out
CNV_AS_SCALE_IN_CONSECUTIVE	4	Number of consecutive low-load intervals required before scale-in
CNV_AS_COOLDOWN_SECONDS	120	Minimum delay between scaling actions
CNV_AS_MIN_WORKERS	1	Minimum number of EC2 workers in the ASG
CNV_AS_MAX_WORKERS	3	Maximum number of EC2 workers in the ASG
CNV_DYNAMIC_COST_MODEL_ENABLED	true	Enables DynamoDB-backed dynamic cost estimation
CNV_COST_HISTORY_LIMIT	200	Maximum number of historical metric records cached by the estimator
CNV_COST_REFRESH_SECONDS	30	Refresh interval for the cached cost history
CNV_METRICS_TABLE	NatureCloudMetrics	DynamoDB table used to store request metrics

Table II. Runtime telemetry summary by workload.

Workload	Records	Median APE	APE $\leq 10\%$	Successes
DNA	7	100.0%	14.3%	7
Fractals	19	72.0%	5.3%	19
Gray-Scott	75	0.0%	96.0%	75

Table III. Selected estimated and measured instruction counts.

Workload	Parameters	Actual instructions	Estimated instructions	APE
Fractals	$w = 800,\ h = 600,\ iter = 100$	6.93×10^{8}	6.93×10^{8}	0.02%
Fractals	$w = 500,\ h = 300,\ iter = 20$	9.80×10^{7}	3.42×10^{7}	65.09%
Gray-Scott	$size = 512,\ iter = 1000$	7.16×10^{10}	6.62×10^{10}	7.62%
Gray-Scott	$size = 512,\ iter = 300$	2.15×10^{10}	2.15×10^{10}	0.00%
DNA	$len_1 = 444,\ len_2 = 374,\ minLength = 5,\ stopOnFirst = false$	1.73×10^{5}	1.73×10^{5}	0.00%
DNA	$len_1 = 454,\ len_2 = 459,\ minLength = 250,\ stopOnFirst = false$	1.17×10^{5}	4.29×10^{5}	266.05%

Table IV. Observed scaling behavior from controller logs.

Time Phase	Target Workers	Active Workers	Avg. Load	AS Decision
Idle	1	1	0	lowIntervals increments steadily while the system remains under the scale-in threshold.
Workload Ingestion	1	1	8.59×10^{11}	A Gray-Scott request is assigned to worker i-0a0c38874dec7c89e, causing the estimated load to spike.
Scale-out Trigger	1	1	8.59×10^{11}	highIntervals reaches the configured threshold, so the auto-scaler requests scale-out from 1 to 2 workers.
Provisioning Lag	2	1	8.59×10^{11}	AWS is booting the new instance. The desired capacity has increased, but the cluster still has only one active worker.
Discovery and Integration	2	2	4.29×10^{11}	New worker i-08f3bbf17d4e32894 is registered. The estimated average load per active worker is approximately halved.

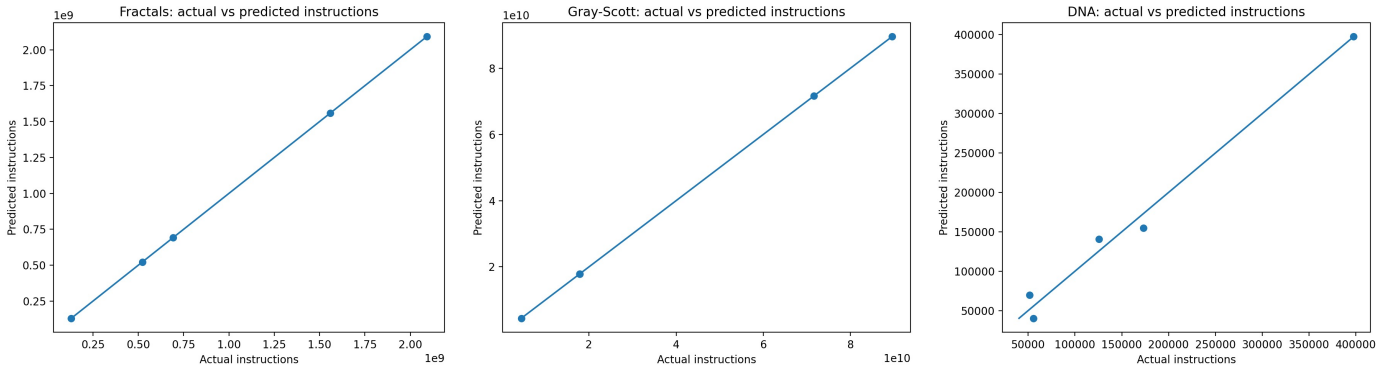


Fig. 1. Actual versus predicted bytecode instruction counts for the three workload-specific fallback models.